

Java Comparator

Rank:

Problem

Submissions

Leaderboard

Discussions

Editorial 100%

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

The `Player` class is provided for you in your editor. It has 2 fields: a `name` String and a `score` integer.

Given an array of `n` `Player` objects, write a comparator that sorts them in order of decreasing score; if 2 or more players have the same score, sort those players alphabetically by name. To do this, you must create a `Checker` class that implements the `Comparator` interface, then write an `int compare(Player a, Player b)` method implementing the `Comparator.compare(T o1, T o2)` method.

Input Format

Input from stdin is handled by the locked stub code in the `Solution` class.

The first line contains an integer, `n`, denoting the number of players.

Each of the `n` subsequent lines contains a player's `name` and `score`, respectively.

Constraints

- $0 \leq \text{score} \leq 1000$
- 2 players can have the same name.
- Player names consist of lowercase English letters.

Output Format

You are not responsible for printing any output to stdout. The locked stub code in `Solution` will create a `Checker` object, use it to sort the `Player` array, and print each sorted element.

Sample Input

Change Theme Language Java 8

```
1 > import java.util.*; ...
12
13 class Checker implements Comparator<Player> {
14     // complete this method
15     public int compare(Player a, Player b) {
16         if (a.score != b.score) {
17             return Integer.compare(b.score, a.score);
18         }
19         // 2. If scores are equal, sort by name alphabetically ascending
20         return a.name.compareTo(b.name);
21     }
22 }
23
24 > ...
```

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

```
1 5
2 amy 100
3 david 100
4 heraldo 50
5 aakansha 75
6 aleksa 150
```

Download

Your Output (stdout)

```
1 aleksa 150
2 amy 100
3 david 100
4 aakansha 75
```

Java Sort ★

Your Java Sort submission got 10.00 points.

[Share](#)

[Post](#)

You are now 15 points away from the 1st star for your java badge

[Try the next challenge](#) | [Try a Random Challenge](#)

Problem

Submissions

Leaderboard

Discussions

Editorial 

You are given a list of student information: ID, FirstName, and CGPA. Your task is to rearrange them according to their CGPA in decreasing order. If two student have the same CGPA, then arrange them according to their first name in alphabetical order. If those two students also have the same first name, then order them according to their ID. No two students have the same ID.

Hint: You can use comparators to sort a list of objects. See the [oracle docs](#) to learn about comparators.

Input Format

The first line of input contains an integer N , representing the total number of students. The next N lines contains a list of student information in the following structure:

```
ID Name CGPA
```

Constraints

$$2 \leq N \leq 1000$$

$$0 \leq ID \leq 100000$$

$$5 \leq |Name| \leq 30$$

$$0 \leq CGPA \leq 4.00$$

```
1  import java.util.*;
2
3  class Student{
4      private int id;
5      private String fname;
6      private double cgpa;
7      public Student(int id, String fname, double cgpa) {
8          super();
9          this.id = id;
10         this.fname = fname;
11         this.cgpa = cgpa;
12     }
13     public int getId() {
14         return id;
15     }
16     public String getFname() {
17         return fname;
18     }
19     public double getCgpa() {
20         return cgpa;
21     }
22 }
23
24 //Complete the code
25 //Complete the code
26 public class Solution
27 {
28     public static void main(String[] args){
29         Scanner in = new Scanner(System.in);
30         int testCases = Integer.parseInt(in.nextLine());
31
32         List<Student> studentList = new ArrayList<Student>();
33         while(testCases>0){
34             int id = in.nextInt();
35             String fname = in.next();
36             double cgpa = in.nextDouble();
37
38             Student st = new Student(id, fname, cgpa);
```

Congratulations

You solved this challenge. Would you like to challenge your friends?

[Next Challenge](#)

✓ Test case 0

✓ Test case 1

✓ Test case 2

✓ Test case 3

✓ Test case 4

✓ Test case 5

✓ Test case 6

Compiler Message

Success

Input (stdin)

```
1 5
2 33 Rumpa 3.68
3 85 Ashis 3.85
4 56 Samiha 3.75
5 19 Samara 3.75
6 22 Fahim 3.76
```

[Download](#)

Expected Output

```
1 Ashis
```

[Download](#)

Sorting: Comparator

Sorting: Comparator ★

Your Sorting: Comparator submission got 35.00 points. [f Share](#) [X Post](#)

[Try the next challenge](#) | [Try a Random Challenge](#)

[Problem](#) | [Submissions](#) | [Leaderboard](#) | [Discussions](#) | [Editorial](#) | [Topics](#)

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array. The Player class is provided in the editor below. It has two fields:

1. **name**: a string.
2. **score**: an integer.

Given an array of n Player objects, write a comparator that sorts them in order of decreasing score. If 2 or more players have the same score, sort those players alphabetically ascending by name. To do this, you must create a Checker class that implements the Comparator interface, then write an `int compare(Player a, Player b)` method implementing the `Comparator.compare(T o1, T o2)` method. In short, when sorting in ascending order, a comparator function returns -1 if $a < b$, 0 if $a = b$, and 1 if $a > b$.

Declare a Checker class that implements the comparator method as described. It should sort first descending by score, then ascending by name. The code stub reads the input, creates a list of Player objects, uses your method to sort the data, and prints it out properly.

Example

$n = 3$ data = `[[Smith, 20], [Jones, 15], [Jones, 20]]`

Sort the list as `data_sorted = [[Jones, 20], [Smith, 20], [Jones, 15]]`. Sort first descending by score, then ascending by name.

Input Format

The first line contains an integer, n , the number of players.

Each of the next n lines contains a player's **name** and **score**, a string and an integer.

Constraints

$0 < n < 1000$

```
1 > import java.util.*; ...
12
13 class Checker implements Comparator<Player> {
14     // complete this method
15     public int compare(Player a, Player b) {
16         if (a.score != b.score) {
17             return Integer.compare(b.score, a.score);
18         }
19         // 2. If scores are equal, sort by name alphabetically ascending
20         return a.name.compareTo(b.name);
21     }
22 }
23
24 > ...
```

Congratulations

You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#)

[Next Challenge](#)

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Test case 6

Compiler Message

Success

Input (stdin)

1 5
2 amy 100
3 david 100
4 heraldo 50
5 aakansha 75
6 aleksa 150

Expected Output

1 aleksa 150

Download

Download

1480. Running Sum of 1d Array

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0..i])`.
Return the running sum of `nums`.

Example 1:
Input: `nums = [1,2,3,4]`
Output: `[1,3,6,10]`
Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:
Input: `nums = [1,2,3,4]`
Output: `[1,3,6,10]`
Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Accepted

Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =
[1,2,3,4]

Output

[1,3,6,10]

Expected

[1,3,6,10]

Problem List

1672. Richest Customer Wealth

Solved

Description

Accepted

1672. Richest Customer Wealth

Solved

Easy

Topics

Companies

Hint

You are given an $n \times m$ integer grid `accounts` where `accounts[i][j]` is the amount of money the i^{th} customer has in the j^{th} bank. Return the **wealth** that the richest customer has.

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum **wealth**.

Example 1:

Input: `accounts = [[1,2,3],[3,2,1]]`

Output: `6`

4.3K 108 43 Online

Code

Java

Auto

```
1 class Solution {
2     public int maximumWealth(int[][] accounts) {
3         int maxWealth = 0;
4
5         // Iterate through each customer (row)
6         for (int[] customer : accounts) {
7             int currentWealth = 0;
8
9             // Sum up money across all banks (columns) for the current customer
10            for (int bank : customer) {
11                currentWealth += bank;
12            }
13
14            // Update the max wealth found so far
15            maxWealth = Math.max(maxWealth, currentWealth);
16        }
17    }
18 }
```

Ln 18, Col 26

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

`accounts =`

`[[1,2,3],[3,2,1]]`

Output

`6`

Expected

`6`

Contribute a testcase

34°C

Mostly cloudy

Search

11:41 AM

28 Jul 28

Problem List < > 74

Accepted X

977. Squares of a Sorted Array

Easy Topics Compare

Given an integer array `nums` sorted in **non-decreasing** order, return an array of the **squares** of each number sorted in **non-decreasing** order.

Example 1:

Input: `nums = [-4,-1,0,3,10]`
Output: `[0,1,9,16,100]`
Explanation: After squaring, the array becomes `[16,1,0,9,100]`. After sorting, it becomes `[0,1,9,16,100]`.

10.4K 194 281 Online

Code

Java Auto

```
1 class Solution {
2     public int[] sortedSquares(int[] nums) {
3         int n = nums.length;
4         int[] result = new int[n];
5
6         int left = 0;
7         int right = n - 1;
8         int index = n - 1; // Start filling result from the back
9
10        while (left <= right) {
11            int leftSquare = nums[left] * nums[left];
12            int rightSquare = nums[right] * nums[right];
13
14            // Compare squares and place the larger one at the current end position
15            if (leftSquare > rightSquare) {
```

Ln 25, Col 23

Testcase > Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =

[-4,-1,0,3,10]

Output

[0,1,9,16,100]

Expected

[0,1,9,16,100]

Contribute a testcase

34°C Mostly cloudy 11:44 AM 28 Jul 26

Problem List < > 74

Accepted X

724. Find Pivot Index

Easy Topics Compare Hint

Given an array of integers `nums`, calculate the **pivot index** of this array.

The **pivot index** is the index where the sum of all the numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right.

If the index is on the left edge of the array, then the left sum is 0 because there are no elements to the left. This also applies to the right edge of the array.

Return the **leftmost pivot index**. If no such index exists, return `-1`.

9.5K 273 151 Online

Code

Java Auto

```
1 class Solution {
2     public int pivotIndex(int[] nums) {
3         int totalSum = 0;
4         int leftSum = 0;
5
6         // Step 1: Calculate the total sum of the array
7         for (int num : nums) {
8             totalSum += num;
9         }
10
11        // Step 2: Iterate and check for the pivot condition
12        for (int i = 0; i < nums.length; i++) {
13            // Check if left sum equals right sum
14            if (leftSum == totalSum - leftSum - nums[i]) {
```

Ln 21, Col 20

Testcase > Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =

[1,7,3,6,5,6]

Output

3

Expected

3

Contribute a testcase

34°C Mostly cloudy 11:47 AM 28 Jul 26